Wireshark Sample Capture Writeup 1

`http_with_jpegs.cap.gz`

Week 2, DG Interns Hub Cyber Security Internship, Network Traffic Analysis

Prepared by: **Harini**

Summary. I analysed this capture to answer three questions: which protocols appear, what happened, and whether anything is suspicious.
Protocols: Ethernet, IPv4, TCP and HTTP.
What happened: a browser opened a small website, followed a link and downloaded pictures. It also contacted an advertising server.
Suspicious: I found no sign of an attack in the packets I examined. The traffic is not encrypted and it contains some IP fragments, so I noted both.

1. About the capture

Item	Detail
Source	Wireshark SampleCaptures wiki, HTTP section. The wiki describes it as a simple capture with a few JPEG pictures that can be put back together and saved to a file.
Tool	Wireshark 4.6.9 on Windows 11.
Size	483 packets.
Date of the traffic	The server's Date header says Sat, 20 Nov 2004 10:21:06 GMT. This is old traffic recorded in 2004.
Hosts	Client: 10.1.1.101. Web server: 10.1.1.1. The client also contacted 209.225.11.237 and 209.225.0.6.

I opened the file in Wireshark and used these tools: the display filters `http`, `!tcp`, `tcp.stream eq 0` and `http.request.method == "POST"`, Statistics, Protocol Hierarchy, Follow TCP Stream, and File, Export Objects, HTTP.

2. Which protocols appear

I opened Statistics, Protocol Hierarchy with no display filter, so it counts all 483 packets (Figure 1).

Protocol	Packets	What it does in this capture
Ethernet and IPv4	483 (100%)	They carry every packet.
TCP	464 (96.1%)	It gives reliable transport. The first three packets are the TCP handshake from 10.1.1.101 to port 80 of 10.1.1.1.
HTTP	39 (8.1%)	It carries the web requests and replies: GET requests, one POST, and 200 OK replies with pages and pictures.
Line-based text data	5	HTML text sent by the web server.
JPEG File Interchange Format	5	Picture files sent by the web server.
Data (IP fragments)	19 (3.9%)	Pieces of larger packets that IP split up. I explain them in Section 4.

No DNS or other protocols appear in the hierarchy. The client already knew the server addresses, so this capture only has web traffic over TCP.

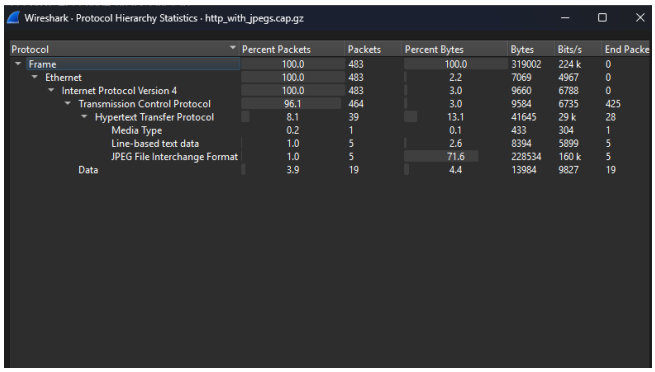


Figure 1. Protocol Hierarchy of the full capture (483 packets, no display filter).

3. What happened

I followed the packets in order. Here is the story in six steps.

Step 1. The browser asks for the home page

The client 10.1.1.101 opened a TCP connection to port 80 of 10.1.1.1. Packets 1 to 3 show the handshake: SYN, SYN-ACK and ACK. In packet 4, the browser sent GET /. In packet 6, the server answered 200 OK with a small HTML page of 160 bytes. I used Follow TCP Stream to read this conversation (Figure 2). The reply came from Apache/2.0.40 (Red Hat Linux), and the browser was Opera 7.11 on Windows. The page only holds a link to ./Websidan/index.html.

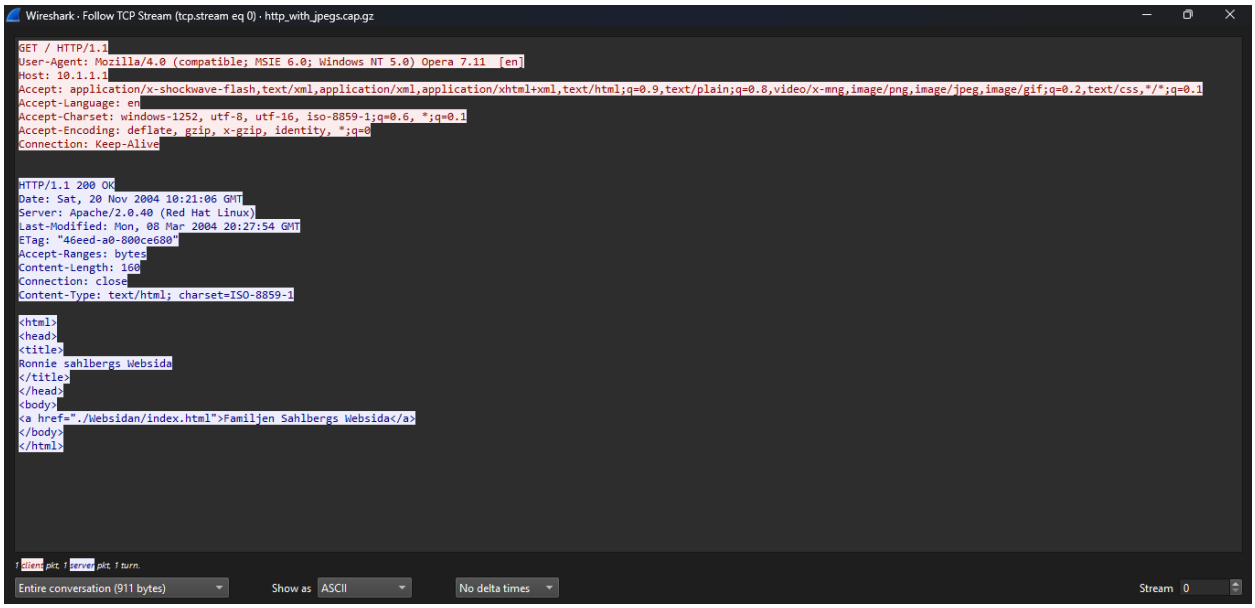


Figure 2. Follow TCP Stream for the first conversation (tcp.stream eq 0): GET / and the 200 OK reply.

Step 2. The browser follows the link and downloads pictures

Next, the browser followed that link. Packets 31 and 38 are the request and the reply for /Websidan/index.html, which is 4323 bytes. The page then loaded two pictures, bg2.jpg and sydney.jpg. The requests are in packets 48 and 50, and the image/jpeg replies are in packets 61 and 72.

Step 3. The browser contacts an advertising server

At the same time, the browser sent a POST in packet 16 to ins1.opera.com, which is server 209.225.11.237. The path is /scripts/cms/xcms.asp. The body is a small XML message. It names the product (Opera, version 711), the language (en) and a user_code number. The server replied with XML that contains advertising links, such as opera3-servedby.advertising.com. Part of this reply is missing from the capture file, and Wireshark shows the note [1460 bytes missing in capture file] (Figure 3).

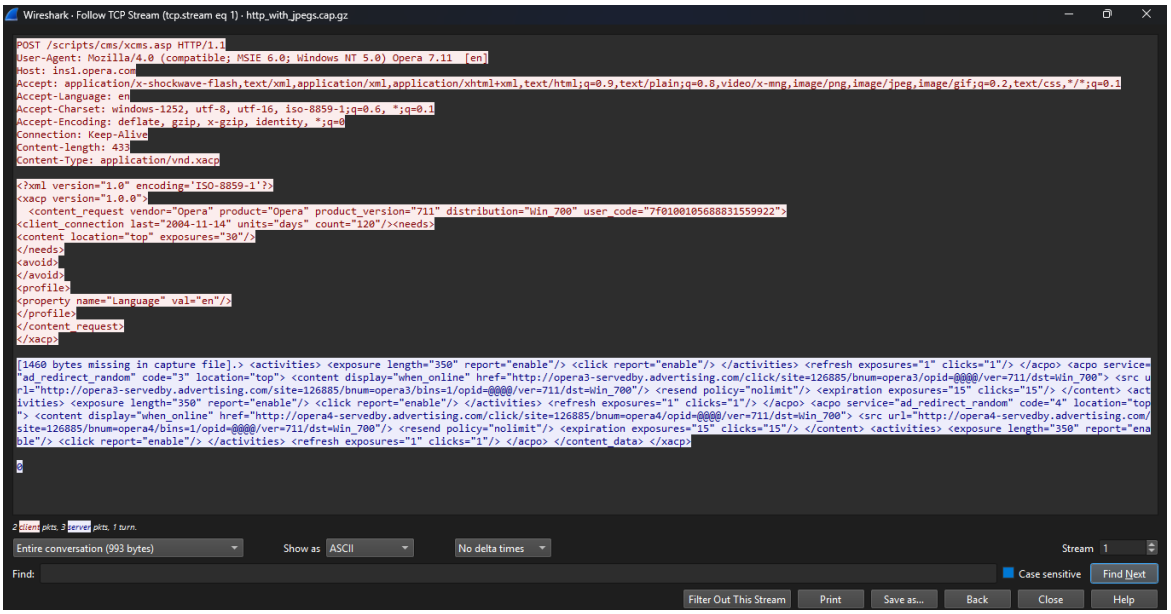


Figure 3. Follow TCP Stream for the POST to ins1.opera.com (tcp.stream eq 1).

Step 4. More requests and more pages

From packet 82, the client sent several GET requests to a third server, 209.225.0.6. Their paths start with /site=126885/bnum=opera, which is the same pattern as the links in the POST reply. So I think these are advert requests. This is my inference, because the capture does not show a host name for this server. Later in the capture, the web server also sent a page called dagbok.html and pictures named DSC07858.JPG and DSC07859.JPG.

Step 5. Listing the files that were sent

To see every file sent over HTTP, I used File, Export Objects, HTTP. The list shows the packet number, host, content type, size and file name of each file (Figure 4).

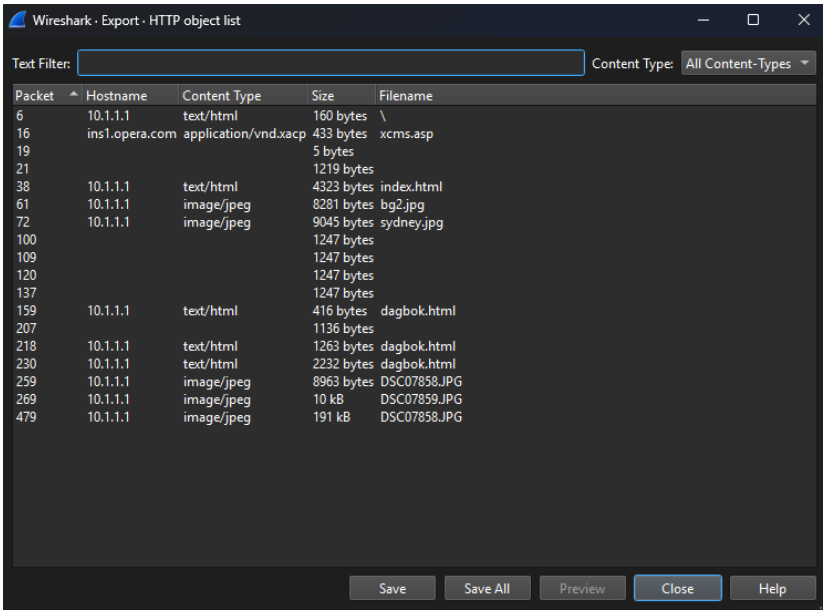


Figure 4. Export Objects, HTTP object list.

Step 6. Rebuilding a picture from the packets

I saved sydney.jpg (packet 72, 9045 bytes) from this list and opened it. It is a small picture, 500 x 89 pixels and 8.8 KB. It shows a city skyline with a harbour, boats and a large bridge (Figure 5). This shows that Wireshark can rebuild a working file from captured packets.

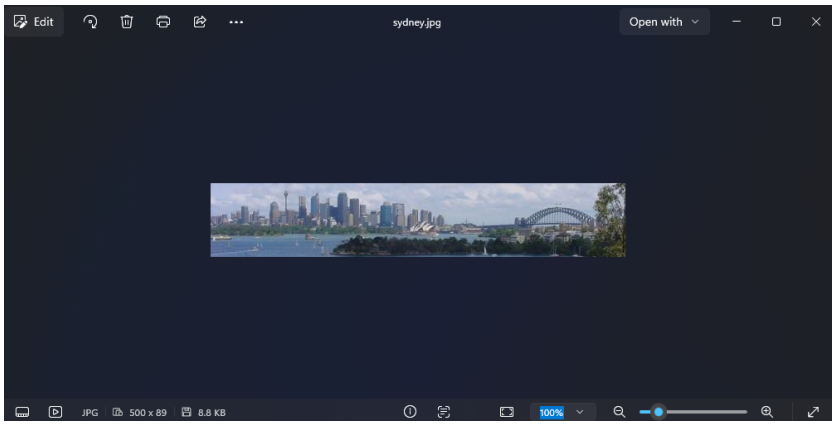


Figure 5. sydney.jpg saved from the capture and opened in the picture viewer.

4. Anything suspicious?

Result: I found no sign of an attack in the packets I examined. The traffic looks like normal web browsing with some advertising traffic. I still noted four points:

- **The traffic is not encrypted.** HTTP sends everything as plain text, so anyone on the network could read the requests, pages and pictures. The POST sends a user_code number, and the User-Agent line shows the browser and the operating system. This is a privacy weakness of HTTP. It is not an attack.
- **IP fragments.** 19 packets (3.9%) are shown as Fragmented IP protocol (Figure 6). Each is 770 bytes and comes from 209.225.11.237 or 209.225.0.6. IP splits a packet into pieces when it is too big for the network, so this is normal. Attackers can also use fragments to hide traffic from security tools, so I checked one. The fragment I opened holds advertising XML, which fits normal web traffic.
- **Missing and out-of-order packets.** Packets 19 and 21 are marked TCP Previous segment not captured and TCP Out-Of-Order. The POST reply also says 1460 bytes are missing. These are gaps in how the sample was recorded, not signs of an attack.
- **Old software.** The server is Apache/2.0.40 (Red Hat Linux) and the browser is Opera 7.11. This fits the 2004 date. The same versions on a network today would be outdated. A server that shows its exact version also helps an attacker choose an exploit.

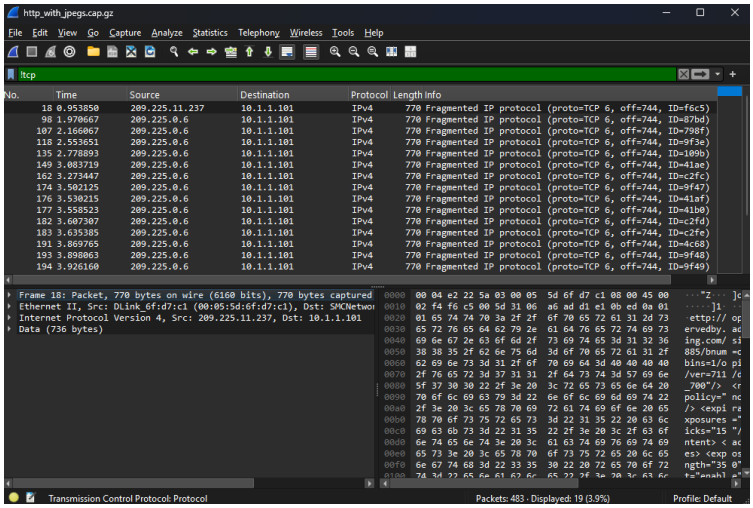


Figure 6. Filter !tcp shows the 19 IP fragments (Displayed: 19, 3.9%).

5. Conclusion

A client at 10.1.1.101 opened a small website on 10.1.1.1. It followed a link to a second page and downloaded the pictures on it. At the same time, the browser asked an advertising server for adverts and received XML in return. The only protocols are Ethernet, IPv4, TCP and HTTP. I found nothing in the examined packets that points to an attack.

What I learned: unencrypted HTTP exposes data to anyone on the network. IP fragments are normal, but I should understand them and not ignore them. Wireshark can rebuild files, such as pictures, from captured packets.

Evidence: Figures 1 to 6 are screenshots I took from this capture in Wireshark. I downloaded the sample capture from the Wireshark SampleCaptures wiki.